

Ensembles

Définition d'un ensemble. Opérations sur les ensembles. Accès, suppression et ajout d'éléments d'un ensemble. Fonctions permettant de manipuler les ensembles.

Les ensembles en Python

- Ce sont des ensembles non ordonnés de valeurs distinctes. Ces valeurs peuvent être de différents types.
- Les tests d'appartenance de même que les opérations d'union et d'intersection fonctionnent comme attendu en Python.
- Il est possible de tester l'appartenance à un ensemble à l'aide des opérateurs `in` et `not in`, d'obtenir le cardinal d'un ensemble à l'aide de la fonction intégrée `len()` et de parcourir les éléments d'un ensemble à l'aide de boucles.
- Puisque les ensembles ne sont pas ordonnés, il n'est pas possible d'utiliser un indice ni de sélectionner une tranche, et il n'existe pas de clé permettant d'accéder aux valeurs.
- Il existe 2 types d'ensembles :
 - `set` : On peut ajouter ou supprimer des éléments à ce type d'ensemble; par conséquent, ils ne peuvent être utilisés comme clés de dictionnaire.
 - `frozenset` : On ne peut ajouter ou supprimer des éléments à ce type d'ensemble; par contre, ils peuvent être utilisés comme clés de dictionnaire ou comme membres d'un autre ensemble.

Symboles d'opérations et de relations s'appliquant aux ensembles

<i>Symbole mathématique</i>	<i>Symbole Python</i>	<i>Description</i>
$\in$	in	Appartient à
$\notin$	not in	N'appartient pas à
$=$	==	Est égal à
$\neq$	!=	N'est pas égal à
$\subset$	<	Est un sous-ensemble strict de
$\subseteq$	<=	Est un sous-ensemble (au sens large de)
$\supset$	>	Est un sur-ensemble strict de
$\supseteq$	>=	Est un sur-ensemble (au sens large de)
$\cap$	&	Intersection
$\cup$	 	Union
$-$ ou $\setminus$	-	Différence ou complément relatif
Δ	^	Différence symétrique

Comment créer des ensembles et leur affecter des valeurs ?

- Il n'existe pas de syntaxe particulière pour les ensembles comme il en existe par exemple pour les listes et pour les dictionnaires.
- La seule méthode de création possible consiste à utiliser les fonctions `set()` et `frozenset()`.

```
>>> S = set("Nous sommes.")
>>> print(S)
set(['o', ' ', 's', 'm', 'u', 'N', 'e'])
>>> len(S)
7
>>> type(S)
<type 'set'>
>>> T = frozenset("Nous avons un ensemble de valeurs distinctes.")
>>> print(T)
frozenset(['a', ' ', 'c', 'b', 'e', 'd', 'i', 'm', 'l', 'o', 'N', '.', 's', 'r', 'u', 't', 'v', 'n'])
>>> len(T)
18
>>> type(T)
<type 'frozenset'>
>>> S == T
False
```

Comment créer des ensembles et leur affecter des valeurs ?

- On peut créer un ensemble à l'aide d'une liste, d'un tuple ou d'un dictionnaire.

```
>>> Voyelles = set(["a", "e", "i", "o", "u", "y"])
>>> print(Voyelles)
set(['a', 'e', 'i', 'o', 'u', 'y'])
```

```
>>> Voyelles = set(("a", "e", "i", "o", "u", "y"))
>>> print(Voyelles)
set(['a', 'e', 'i', 'o', 'u', 'y'])
```

```
>>> Voyelles = set({0:"a", 1:"e", 2:"i", 3:"o", 4:"u", 5:"y"})
>>> print(Voyelles)
set([0, 1, 2, 3, 4, 5])
>>> Voyelles = set({0:"a", 1:"e", 2:"i", 3:"o", 4:"u", 5:"y"}.items())
>>> print(Voyelles)
set([(0, 'a'), (2, 'i'), (1, 'e'), (5, 'y'), (4, 'u'), (3, 'o')])
>>> Voyelles = set({0:"a", 1:"e", 2:"i", 3:"o", 4:"u", 5:"y"}.keys())
>>> print(Voyelles)
set([0, 1, 2, 3, 4, 5])
```

- Si aucun argument n'est fourni, il y a création d'un ensemble vide. →

```
>>> set()
set([])
>>> set([])
set([])
>>> set(())
set([])
```

Comment modifier un ensemble ?

- 1^{ière} façon : ajouter un élément à un ensemble (**add**)

```
>>> voyelles = set("aeiou")
>>> voyelles.add("y")
>>> print (voyelles)
set(['a', 'e', 'i', 'o', 'u', 'y'])
```

- 2^{ème} façon : ajouter des éléments à un ensemble (**update**)

```
>>> voyelles_et_chiffres = set("aeiouy")
>>> voyelles_et_chiffres.update("0123456789")
>>> print (voyelles_et_chiffres)
set(['a', '9', 'e', 'i', 'o', '1', '0', '3', '2', 'u', '4', '7', '6', 'y', '5', '8'])
>>>
>>> voyelles_et_chiffres = set("aeiouy")
>>> voyelles_et_chiffres.update(range(10))
>>> print(voyelles_et_chiffres)
set(['a', 0, 2, 3, 'e', 5, 6, 1, 'i', 9, 7, 'o', 8, 'u', 'y', 4])
>>>
>>> voyelles_et_chiffres = set("aeiouy")
>>> voyelles_et_chiffres.update([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> print(voyelles_et_chiffres)
set(['a', 0, 2, 3, 'e', 5, 6, 1, 'i', 9, 7, 'o', 8, 'u', 'y', 4])
```

Comment modifier un ensemble ?

- 3^{ème} façon : enlever un élément faisant partie d'un ensemble (**remove**),
enlever un élément à un ensemble s'il en fait partie (**discard**),
enlever et retourner un élément quelconque d'un ensemble (**pop**).

```
>>> chiffres = set(range(11))
>>> print(chiffres)
set([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
>>> chiffres.remove(10)
>>> print(chiffres)
set([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

- 4^{ème} façon : enlever des éléments à un ensemble (**-=**)

```
>>> print(chiffres)
set([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> chiffres -= set([4, 6, 8, 9])
>>> print chiffres
set([0, 1, 2, 3, 5, 7])
```

Comment supprimer un ensemble ?

```
Del(chiffres)
```

On peut aussi supprimer tous les éléments d'un ensemble avec la méthode `clear()`.

Opérateurs sur les ensembles de même type ou de types différents

Appartenance (in, not in)

Sert à déterminer si un élément est ou n'est pas membre d'un ensemble.

```
>>> Prenoms = set(['« Bon', « Jour'])  
>>> "Jour" in Prenoms  
True
```

Égalité / inégalité d'ensembles

Sert à tester l'égalité entre ensembles, c'est-à-dire si chaque élément de chaque ensemble est aussi élément de l'autre ensemble. Les ensembles peuvent être de même type ou de types différents.

```
>>> s = set(range(5,10))  
>>> t = frozenset([5, 6, 7, 8, 9])  
>>> s == t  
True  
>>> set("soir") != set("rois")  
False
```

Sous-ensemble de ou sur-ensemble de (<, <=, >, >=)

Inégalité stricte :

$A < B$ signifie que A est un sous-ensemble de B mais n'est pas égal.

$A > B$ signifie que A est un sur-ensemble de B mais n'est pas égal.

Inégalité au sens large :

$A \leq B$ signifie que A est un sous-ensemble de B mais les 2 ensembles peuvent être égaux.

$A \geq B$ signifie que A est un sur-ensemble de B mais les 2 ensembles peuvent être égaux.

```
>>> groupeA = set([" AB", "CD", « EF"])
>>> groupeB = set([" EF", "AB", "CD", "MN", "FG"])
>>> groupeA < groupeB
True
```

Union (|)

Cette opération est équivalente au OU logique entre ensembles. L'union de 2 ensembles est un autre ensemble dont chaque élément appartient au moins à l'un des 2 ensembles. `union()` est une méthode équivalente à cette opération.

```
>>> s = set("aeiouy") | set(range(10))
>>> print (s)
set([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 'a', 'e', 'i', 'o', 'u', 'y'])
>>> set("aeiouy").union(set(range(10)))
set([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 'a', 'e', 'i', 'o', 'u', 'y'])
```

Intersection (&)

Cette opération est équivalente au ET logique entre ensembles. L'intersection de 2 ensembles est un autre ensemble dont chaque élément appartient aux 2 ensembles de départ. `intersection()` est une méthode équivalente à cette opération.

```
>>> set(range(10)) & set(range(5, 15))  
set([8, 9, 5, 6, 7])
```

Différence ou complément relatif (-)

La différence de 2 ensembles est un autre ensemble dont chaque élément appartient au premier ensemble mais pas au second. `difference()` est une méthode équivalente à cette opération.

```
>>> set("proximités") - set("permissivité")  
set(['x', 'o'])
```

Différence symétrique (^)

Cette opération est équivalente à un OU exclusif entre ensembles. La différence symétrique de 2 ensembles est un autre ensemble dont chaque élément appartient à l'un des ensembles mais pas à l'autre. `symmetric_difference()` est une méthode équivalente à cette opération.

```
>>> set("proximités") ^ set("permissivité")  
set(['e', 'o', 'v', 'x'])
```

Opérateurs s'appliquant uniquement aux ensembles de type set seulement

- |=** Cet opérateur ajoute un ou plusieurs membres d'un autre ensemble à l'ensemble existant. `update()` est une méthode équivalente à cette opération.

```
>>> s = set("proximités")
>>> s |= set("permissivité")
>>> print(s)
set(['é', 'e', 'i', 'm', 'o', 'p', 's', 'r', 't', 'v', 'x'])
```

- &=** Cet opérateur ne conserve que les membres de l'ensemble existant qui sont également membres de l'autre ensemble. `intersection_update()` est une méthode équivalente à cette opération.

```
>>> s = set("proximités")
>>> s &= set("permissivité")
>>> print(s)
set(['p', 'é', 'm', 'i', 's', 'r', 't'])
```

- =** Cet opérateur ne conserve que les membres de l'ensemble existant qui ne sont pas membres de l'autre ensemble. `difference_update()` est une méthode équivalente à cette opération.

```
>>> s = set("proximités")
>>> s -= set("permissivité")
>>> print(s)
set(['x', 'o'])
```

$\wedge=$ Cet opérateur ne conserve que les membres appartenant à l'un des deux ensembles d'origine mais pas aux deux. `symmetric_difference_update()` est une méthode équivalente à cette opération.

```
>>> s = set("proximités")
>>> s ^= set("permissivité")
>>> print s
set(['e', 'o', 'v', 'x'])
```

Fonctions intégrées

`len()` Retourne la cardinalité de l'ensemble.

```
>>> len(set("0123456789"))
10
>>> len(range(10))
10
>>> len(set(["a", "e", "i", "o", "u", "y"]))
6
```

- Nous avons déjà mentionné l'existence de plusieurs méthodes s'appliquant aux ensembles équivalentes à des opérateurs.

Il existe une méthode qui ne possède pas d'opérateur équivalent : `copy()`; elle retourne un nouvel ensemble qui est une copie de l'ensemble existant.

```
>>> s
set(['e', 'o', 'v', 'x'])
>>> t = s.copy()
>>> print t
set(['x', 'e', 'o', 'v'])
>>> t.add("u")
>>> print s
set(['e', 'o', 'v', 'x'])
>>> print t
set(['x', 'e', 'u', 'o', 'v'])
```